

Universidade Federal de Mato Grosso – UFMT
Campus Universitário de Várzea Grande
Faculdade de Engenharia
Engenharia de Computação

Carlos Daniel Santana Cunha

Luciano Tadeu Simões Simplicio Junior
Nome3

Relatório
Pathfinding e Otimização de Energia: Uma Abordagem Interativa com o Algoritmo A*

Várzea Grande
18 de outubro de 2026

Carlos Daniel Santana Cunha

Luciano Tadeu Simões Simplicio Junior

Nome3

Relatório
Pathfinding e Otimização de Energia: Uma Abordagem Interativa com o Algoritmo A*

Relatório elaborado como parte das atividades da disciplina Inteligência Artificial, do curso de Bacharel em Engenharia de Computação, oferecida pela Faculdade de Engenharia.
Docente: Dr. Gustavo Post Sabin

Várzea Grande
18 de outubro de 2026

Lista de ilustrações

Figura 1 – *Grid* do Jogo 2
Figura 2 – Classe Estado em *GDScript* 4
Figura 3 – Função A* sem cálculo de estatísticas em *GDScript* 5
Figura 4 – Função Reconstruir Caminho em *GDScript* 6
Figura 5 – Título da figura 7

Lista de tabelas

Tabela 1 – Tabela de custos associados aos terrenos do ambiente virtual. 1
Tabela 2 – Título da tabela 7

Sumário

1 INTRODUÇÃO 1

2 METODOLOGIA 3
2.1 Representação do Problema 3
2.2 Definição 3
2.3 Aplicação da Heurística 4
2.4 A* (A-Estrela) 5

3 RESULTADOS 7

4 CONCLUSÃO 8

REFERÊNCIAS 9

Capítulo 1

Introdução

O problema de busca de rotas ótimas, amplamente conhecido na ciência da computação como *pathfinding*, representa uma das categorias mais clássicas e fundamentais dos problemas de otimização. Desde a roteirização de pacotes em redes de computadores até a navegação de veículos autônomos e a movimentação de personagens em jogos digitais (*NPCs*), a necessidade de deslocar um agente de um ponto de origem a um ponto de destino gastando a menor quantidade de recursos possível é um desafio constante e de alta relevância prática.

No escopo deste trabalho, o problema de otimização escolhido consiste na navegação espacial em um ambiente bidimensional (malha ou *grid*) onde o custo de deslocamento não é homogêneo. Diferentemente de cenários simplificados onde a distância métrica é o único fator limitante, o ambiente modelado apresenta diferentes tipos de superfícies e obstáculos, cada um impondo uma penalidade distinta ao deslocamento do agente, definida como "Custo de Energia". O objetivo central do problema é encontrar um caminho válido que conecte a origem ao destino minimizando o somatório total de energia gasta ao longo do trajeto.

Para ilustrar, analisar e testar a resolução matemática deste problema, foi desenvolvida uma aplicação interativa estruturada como um jogo de quebra-cabeça (*puzzle*) em 2D. A topologia do ambiente é composta por terrenos modulares, cujos pesos foram definidos da seguinte forma:

Tipo de Terreno	**Características**	**Custo de Energia**
Grama	Superfície plana e de fácil locomoção	1
Morro	Elevação que exige maior esforço físico	2
Água	Superfície densa e de movimentação lenta	3

Tabela 1 – Tabela de custos associados aos terrenos do ambiente virtual.

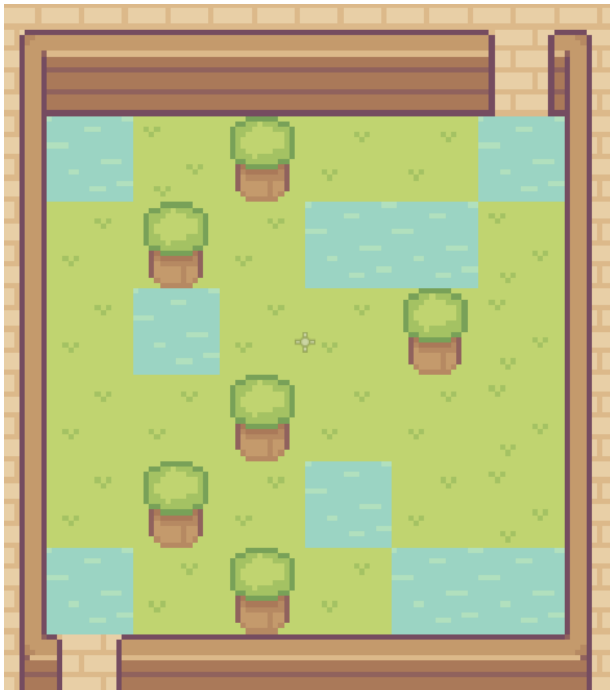


Figura 1 – *Grid* do Jogo

Fonte: Elaborado pelos autores (2026).

A principal contribuição e diferencial da modelagem proposta é a análise comparativa entre a intuição humana e a precisão algorítmica. Na aplicação desenvolvida, o usuário humano é desafiado a traçar, de forma manual e baseada em sua percepção visual, o caminho que julga consumir a menor quantidade de energia. Após a tentativa humana, o sistema computacional resolve o mesmo mapa utilizando o algoritmo heurístico A* (A-Estrela), revelando a rota matematicamente ótima (*gabarito*).

Essa abordagem interativa permite comparar a intuição do jogador contra a precisão matemática da máquina. Ao medir a diferença de custo entre a rota escolhida no 'olhômetro' pelo humano e o gabarito perfeito gerado pelo A*, o trabalho prova na prática por que algoritmos de busca são fundamentais para não desperdiçar recursos em problemas de otimização.

Capítulo 2

Metodologia

A forma de implementação utilizada é o algoritmo de busca A* (A-Estrela), a forma de solução mais conhecida da busca de melhor escolha, que minimiza o custo total estimado da solução. As seções a seguir explicam as fórmulas do algoritmo, com base em Russell e Norvig (2013, seção 3.5.2).

2.1 Representação do Problema

O ambiente do jogo é uma grade, implementada como uma matriz bidimensional 6×6, com 36 células ou terrenos (malha ou *grid*), como na Figura 1. Cada célula da grade corresponde a um estado, que armazena a posição (x, y), com movimentação de célula em célula (acima, abaixo, esquerda e direita). O custo de cada passo é o "Custo de Energia" da célula onde entra (Grama 1, Morro 2, Água 3), na Tabela 1. A fórmula de somatório do "Custo de Energia" é

$$\sum_{i=0}^{k} c(v_i) \tag{1}$$

onde v_0 é a entrada, v_k é a saída e $c(v_i)$ é o custo de cada célula do caminho. A entrada é a célula $(0, 5)$, onde o A* começa, e a saída é a célula $(5, 0)$.

2.2 Definição

Foi implementado o estado do jogo para o núcleo do algoritmo A* (A-Estrela). Foram definidas as seguintes variáveis, como na Figura 2:

- ❏ **Pai / predecessor π:** guarda uma referência ao estado para reconstruir o caminho do estado de saída de volta até o estado de entrada;
- ❏ **$g(v)$:** é o custo real acumulado do estado de entrada até o estado v;
- ❏ **$h(v)$:** é a heurística para estimar a distância do estado v até o estado de saída, utilizando a distância de Manhattan;

- **$f(v)$:** é a prioridade do estado, o custo estimado da solução de menor gasto de energia que passa por v, a fórmula:

$$f(v) = g(v) + h(v) \tag{2}$$

Figura 2 – Classe Estado em *GDScript*

```
class_name Estado

var posicao: Vector2i ## (x, y)
var pai: Estado ## predecessor π

var g := 0 ## custo do caminho g(v)
var h := 0 ## heurística h(v) ⇒ Manhattan

var f: int: ## prioridade (f(v) = g(v) + h(v))
    get():
        return g + h

func _init(p: Estado, pos: Vector2i) → void:
    pai = p
    posicao = pos

func vizinhos(matriz: Array) → Array[Vector2i]:
    var adj: Array[Vector2i] = [Vector2i(0, -1), Vector2i(0, 1), Vector2i(-1, 0), Vector2i(1, 0)]
    var res: Array[Vector2i] = []

    for direcao in adj:
        var nova_pos := posicao + direcao

        if (nova_pos.x ⩾ 0 and nova_pos.x < matriz[0].size()) \
        and (nova_pos.y ⩾ 0 and nova_pos.y < matriz.size()):
            res.append(nova_pos)

    return res

func manhattan(pos: Vector2i, obj: Vector2i) → int:
    return abs(obj.x - pos.x) + abs(obj.y - pos.y)
```

Fonte: Elaborado pelos autores (2026).

2.3 Aplicação da Heurística

A heurística escolhida é a distância de Manhattan no algoritmo A*, a soma das distâncias da posição até a saída. O movimento é proibido ao longo de diagonais, então a distância contada é a soma das horizontais e verticais:

$$h(v) = |x_s - x_v| + |y_s - y_v| \tag{3}$$

onde (x_v, y_v) é a posição do estado v e (x_s, y_s) é a posição do estado de saída. Essa heurística é otimista (RUSSELL; NORVIG, 2013, seção 3.6).

2.4 A* (A-Estrela)

O algoritmo foi adaptado, conforne a Figura 3, para que o custo inicial g do estado de entrada corresponda ao valor da célula de partida, conforme as regras do jogo:

Figura 3 – Função A* sem cálculo de estatísticas em *GDScript*

```
static func astar(matriz: Array, entrada := Vector2i(0, 5), saida := Vector2i(5, 0)) -> Estado:
    var Q := FilaPrioridade.new()
    var visitada := {}
    var g_values := {}

    var start := Estado.new(null, entrada)
    start.g = matriz[entrada.y][entrada.x]
    start.h = start.manhattan(entrada, saida)
    g_values[entrada] = start.g # Inicializar custo do estado entrada

    Q.enfileirar(start, start.f)

    while Q.contar > 0: # verifica se Q está vazia
        var v: Estado = Q.desenfileirar()

        if v.posicao in visitada:
            continue

        visitada[v.posicao] = true

        if v.posicao == saida:
            return v

        for u in v.vizinhos(matriz):
            var novo_g: int = g_values[v.posicao] + matriz[u.y][u.x] # Custo do novo caminho (g(n))

            if u not in g_values or novo_g < g_values[u]:
                g_values[u] = novo_g
                var filho := Estado.new(v, u)
                filho.g = novo_g
                filho.h = filho.manhattan(u, saida)
                Q.enfileirar(filho, filho.f)

    return null
```

Fonte: Elaborado pelos autores e adaptado do Laboratório 3 da disciplina (2026).

- ❑ **Fila de prioridade:** foi implementada a fila de prioridade, pois a linguagem GDScript (Godot) não possui uma estrutura pronta. Ela possui as funções enfileirar, que armazena o estado na posição certa pela prioridade $f(v)$, e desenfileirar, que retira o primeiro da fila, o de menor $f(v)$. Na busca A*, utiliza-se f em vez de g para ordenar a fila de prioridade (RUSSELL; NORVIG, 2013, seção 3.5).

- ❑ **Início:** o estado de entrada, com g igual ao custo da célula, h pela distância de Manhattan, e entra na fila.

- ❑ **Repetição:** enquanto a fila não está vazia, retira o estado de menor f.

  - Se já foi visitado, descarta.
  - Se é a saída, termina.

- ❏ **Vizinhos:** para cada vizinho u, o novo custo é $g(v) + c(u)$. Se for menor que o custo já conhecido, guarda o novo custo e coloca u na fila.
- ❏ **Reconstruir caminho:** a função recebe o estado final e reconstrói caminho pelo vetor de pai (predecessor π). Um laço percorre os estados da saída até a entrada, e a função retorna o caminho invertido, conforme a Figura 4.

Figura 4 – Função Reconstruir Caminho em *GDScript*

```gdscript
static func reconstruir_caminho(estado_final: Estado) -> Array[Vector2i]:
    var caminho: Array[Vector2i] = []
    var atual: Estado = estado_final

    while atual != null:
        caminho.append(atual.posicao)
        atual = atual.pai

    caminho.reverse()
    return caminho
```

Fonte: Elaborado pelos autores (2026).

Capítulo 3

Resultados

Figura 5 – Título da figura



Fonte: Elaborado pelos autores (2026).

Tabela 2 – Título da tabela

Coluna 1	Coluna 2	Coluna 3
Linha 1	0	0
Linha 2	0	0

Fonte: Elaborado pelos autores (2026).

Capítulo 4

Conclusão

Referências

RUSSELL, S.; NORVIG, P. **Inteligência artificial**. 3. ed. Rio de Janeiro: Elsevier, 2013. ISBN 978-8535237016.